LYKN — Capture The Flag Write-up

OSINT, Cryptography, Miscellaneous & Reverse Engineering

Contents

1 Challenge 1 — Remedy **2**

2 Challenge 2 — Static **4**

3 Challenge 3 — Miss My School **9**

4 Challenge 4 — Unnamed Merchant **11**

5 Challenge 5 — Important Debris **13**

6 Challenge 6 — Hash & Dash **16**

7 Challenge 7 — Route to Nowhere **22**

8 Challenge 8 — Waguri2 **27**

9 Challenge 9 — Inferior Student **33**

10 Challenge 10 — Far Away **42**

11 Challenge 11 — Far Away Revenge **44**

12 Challenge 12 — Sleepless Machine **47**

Challenge 1 — Remedy

Category

Miscellaneous / Forensics

Problem Description

This is one of the basic challenge for hiding the data in the form of metadata of an image.

Figure 1: Challenge description for Remedy.

Figure 2: Challenge attachment for Remedy.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. `exiftool challenge.png` — Used this command for getting all the metadata. Found the description a bit suspicious — `6d14166842b6ecb67622284a65bde8a87e03344564bde3ab7e1e324b648dc4a87e0a2f4976bdffbd7e0233435ea6cbb45c`.

Step 2. Thought of the XOR encryption, using the first 8 blocks of the text as the XOR for the letter 'LYKNCTF', found a repeating pattern and the key: `21 4d 5d 26 01 e2 aa cd`

Step 3. Applying that exact 8-byte key across the rest of the hexadecimal string reveals the plaintext message block-by-block:

- Block 1: `6d14166842b6ecb6` → `LYKNCTF{`
- Block 2: `7622284a65bde8a8` → `Would_Be`
- Block 3: `7e03344564bde3ab` → `_Nice_If`
- Block 4: `7e1e324b648dc4a8` → `_Someone`
- Block 5: `7e0a2f4976bdffbd` → `_Grow_Up`
- Block 6: `7e0233435ea6cbb4` → `_One_Day`
- Final Byte: `5c` → `}`

Flag: `LYKNCTF{Would_Be_Nice_If_Someone_Grow_Up_One_Day}`

Challenge 2 — Static

Category

Miscellaneous / OSINT

Problem Description

This is one of the challenge of analyzing the video and checking for the hidden information being mentioned in the same, in the form of some sort of symbol of another language.

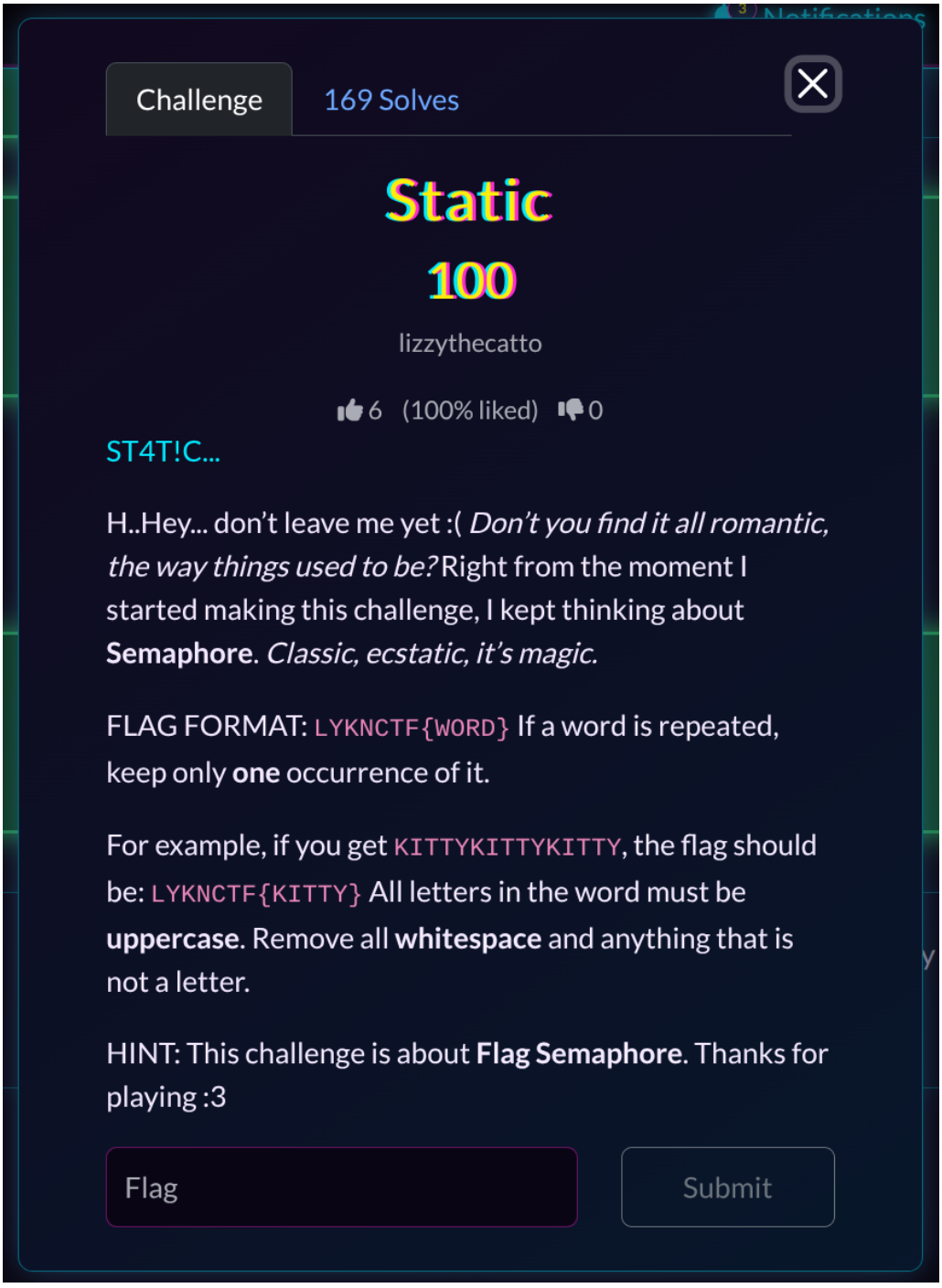


Figure 3: Challenge description for Static.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. https://www.youtube.com/watch?v=KlTNKOnfXFk — Watched the video at the mentioned link, and found that the character is using the FLAG SEMAPHORE for giving some message.

Step 2. The video from 2:31 – 2:33, is depicting word that is getting repeated in the next 3 second duration, as the challenge description said grab the word that is being getting repeated. The word was — DONTGO.

Step 3. Used the mentioned below screenshots for the detection of the word.

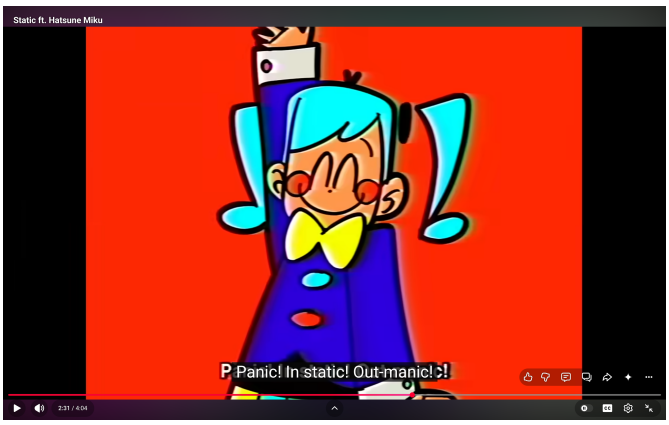


Figure 4: Semaphore screenshot 1.

Figure 5: Semaphore screenshot 2.

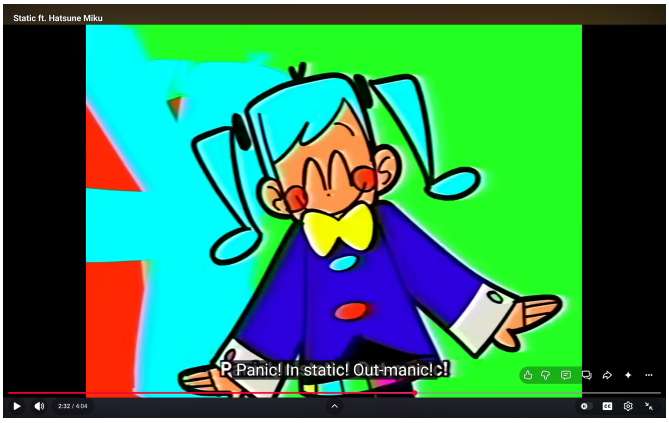


Figure 6: Semaphore screenshot 3.

Figure 7: Semaphore screenshot 4.

Figure 8: Semaphore screenshot 5.

Figure 9: Semaphore screenshot 6.

Figure 10: Semaphore screenshot 7 (combined sequence, part 1).

Figure 11: Semaphore screenshot 8 (combined sequence, part 2).

Figure 12: Semaphore screenshot 9 (combined sequence, part 3).

Figure 13: Semaphore screenshot 10 (combined sequence, part 4).

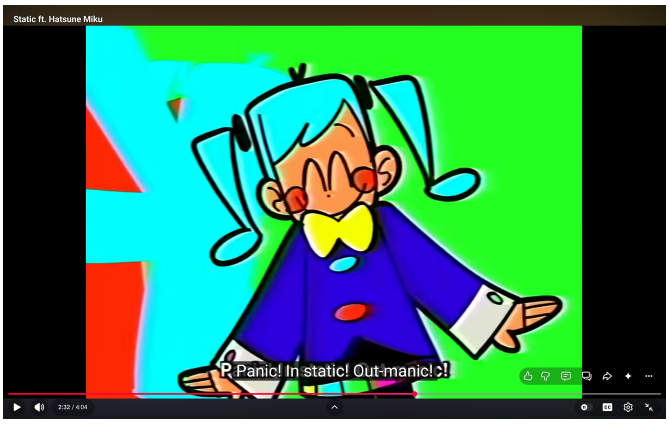


Figure 14: Semaphore screenshot 11 (combined sequence, part 5).

Figure 15: Semaphore screenshot 12 (combined sequence, part 6).

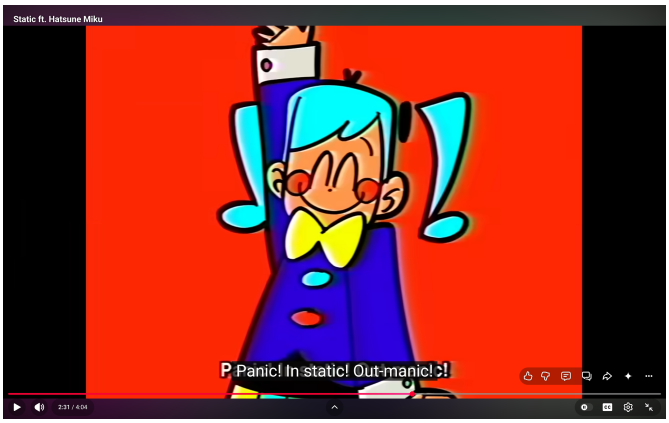


Figure 16: Combined semaphore screenshots showing the repeated word “DONTGO”.

Flag: `LYKNCTF{DONTGO}`

Challenge 3 — Miss My School

Category

OSINT (Open Source Intelligence)

Problem Description

This is one of the classic OSINT challenge for the finding of the name of the school, in which the author of the challenge use to study, and the image of the school is being shared as an attachment: -

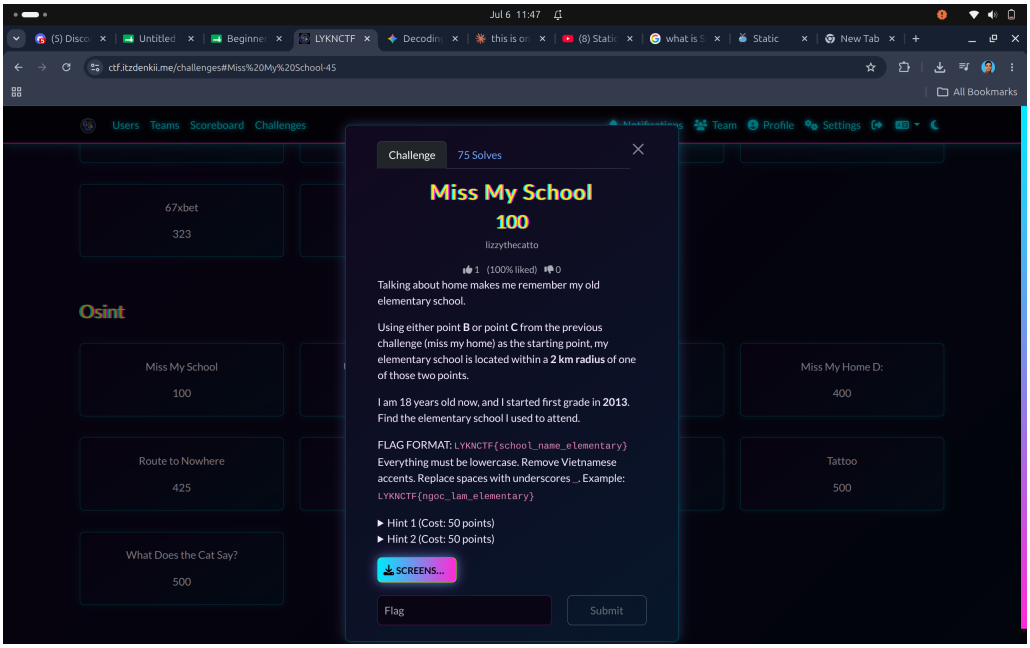


Figure 17: Challenge description for Miss My School.

Figure 18: Image of the school attached with the challenge.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Uploaded the mentioned above picture on GOOGLE IMAGE for finding the similar images, found that the name of the school is - Trường Tiểu học Long Biên

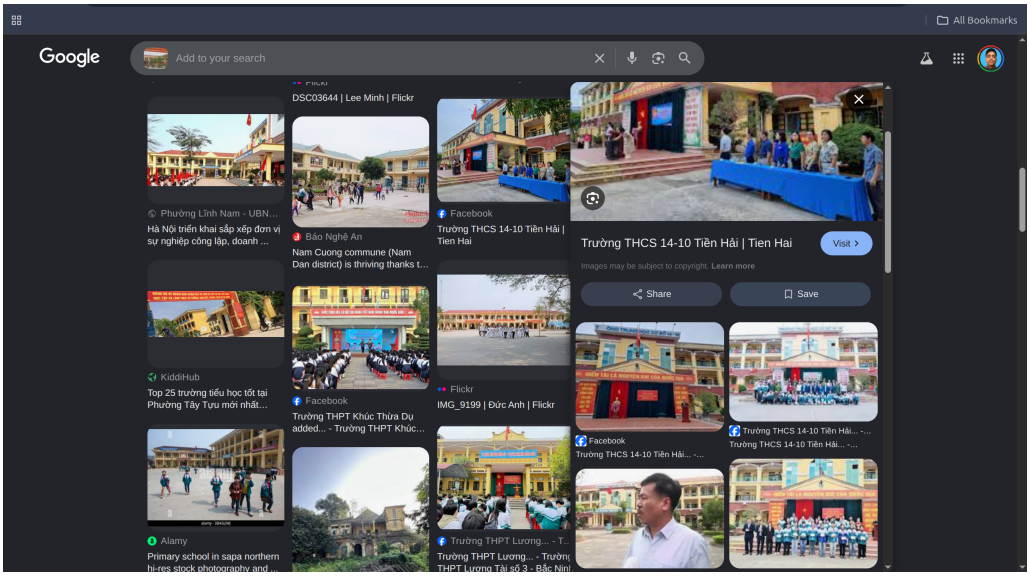


Figure 19: Google Image search result confirming the school name.

Flag: `LYKNCTF{long_bien_elementary}`

Challenge 4 — Unnamed Merchant

Category

OSINT (Open Source Intelligence)

Problem Description

This is one of the classic OSINT challenge for finding some of the information about a material or an object, whose description is present on the internet. In this challenge we have to find a civilian vessel that is being mentioned in the records of AMSA's public MH370 timeline records, we have to find the vessel that was the closest to the Southern Indian Ocean, during the Indian Ocean Search Phase.

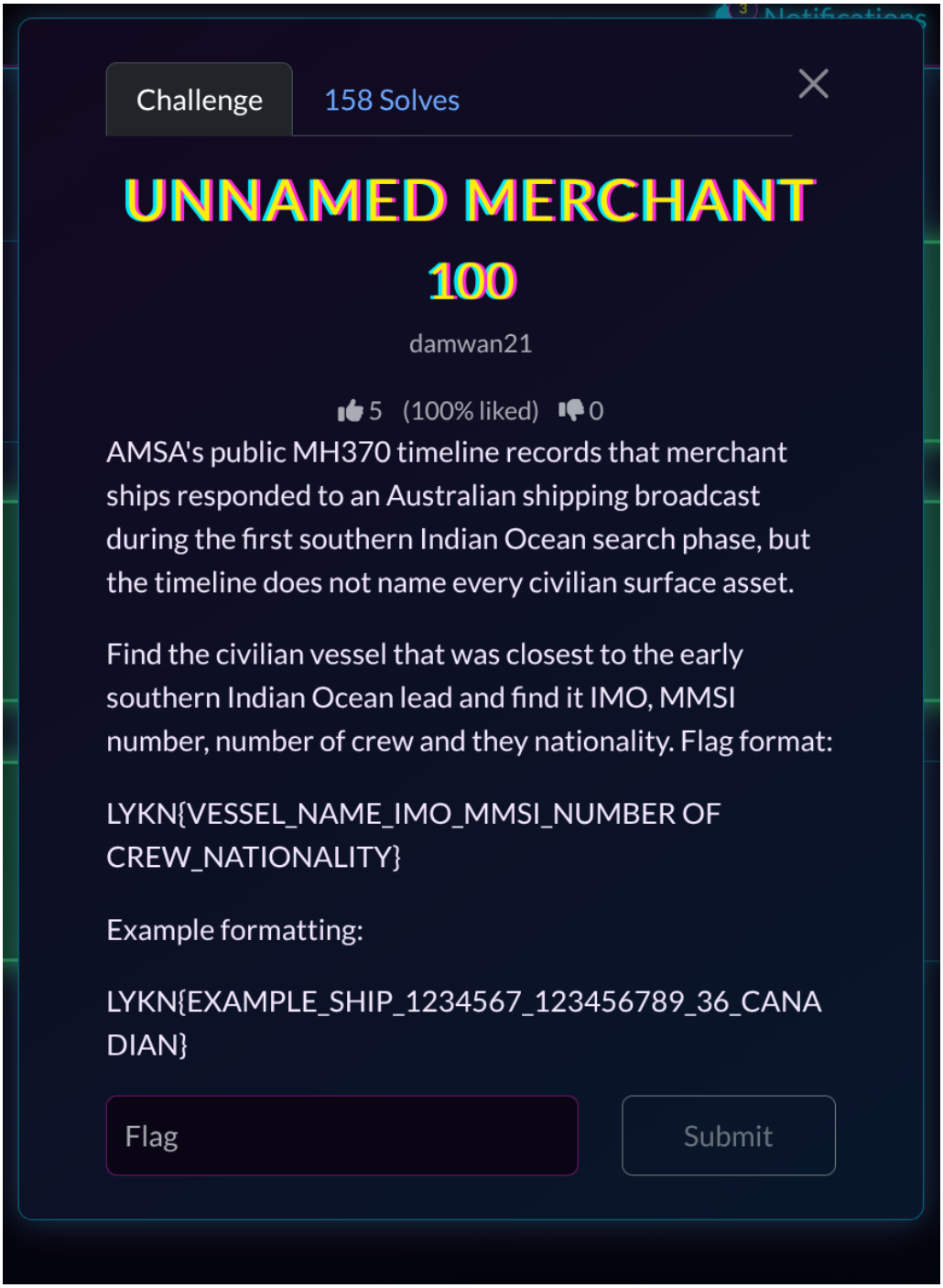


Figure 20: Challenge description for Unnamed Merchant.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Used the LLM for the finding of the information regarding the same description and looked for some of the reports being mentioned online. Found all the required credentials for the flag.

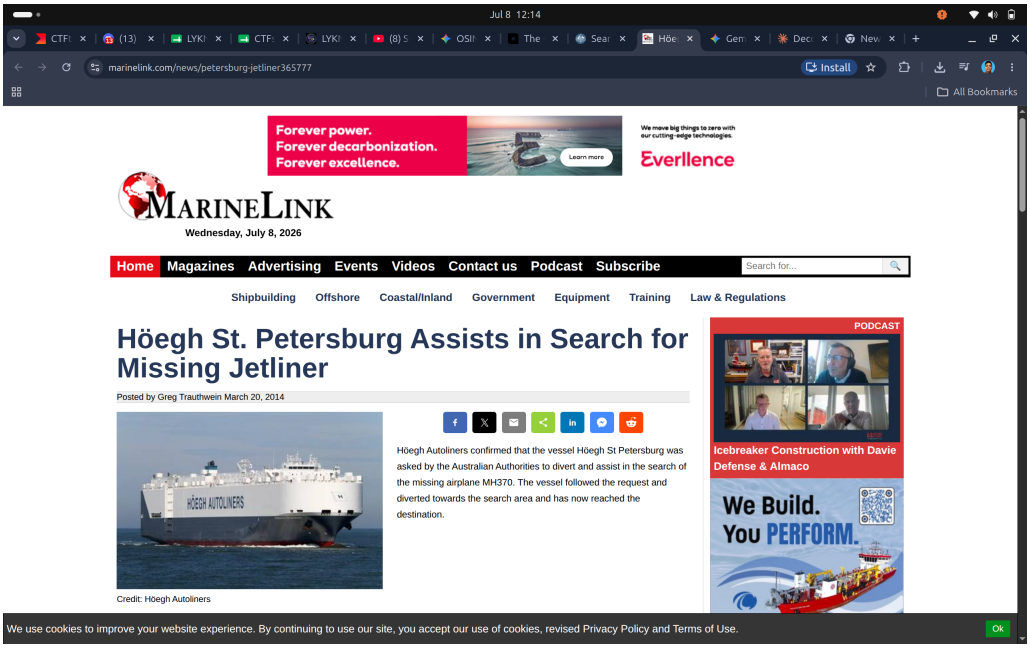


Figure 21: LLM-assisted research into the AMSA MH370 timeline records.

The link for the final report of the search - https://www.atsb.gov.au/sites/default/files/media/5773565/operational-search-for-mh370_final_3oct2017.pdf

Used the mentioned below links also: -

- https://www.offshore-energy.biz/still-no-sign-of-the-missing-flight-mh370/
- http://cbsnews.com/news/malaysia-airlines-flight-370-search-equipment-ranges-from-sophisticated-to-simple/
- https://www.theguardian.com/world/2014/mar/20/mh370-two-possible-objects-may-have-been-found-in-australian-search-zone

Step 2. Formed all the credentials in the required order and got the flag.

- Vessel Name - HOEGH_ST_PETERSBURG
- IMO - 9420045
- MMSI - 257366000
- Number of Crew — 19
- Nationality - FILIPINO

Flag: `LYKN{HOEGH_ST_PETERSBURG_9420045_257366000_19_FILIPINO}`

Challenge 5 — Important Debris

Category

OSINT (Open Source Intelligence)

Problem Description

This is one of the classic OSINT challenge for finding some of the information about a material or an object, whose description is present on the internet. In this challenge, we are needed to find the information about a derbis report which describes several pieces from Madagascar, and one item stands out because investigators could link a small marking to a Boeing cabin component. We have to find the information about the same piece.

Figure 22: Challenge description for Important Debris.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Used the LLM for the finding of the information regarding the same description and looked for some of the reports being mentioned online. Found all the required credentials for the flag.

Mentioned below is the image of the report that is being used for the investigation of the challenge, with the URL.

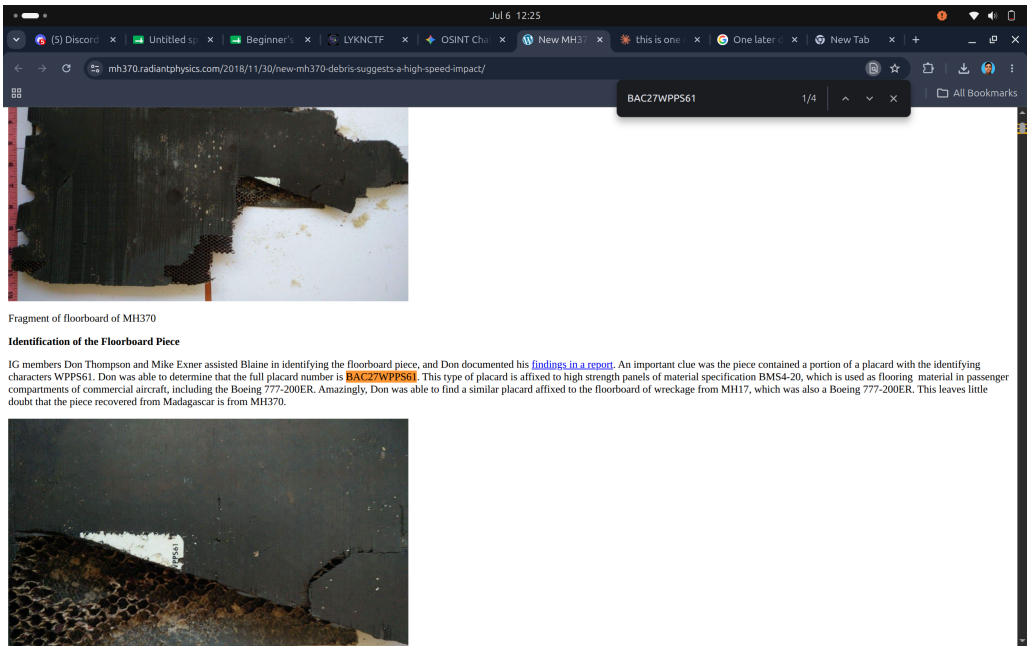


Figure 23: Debris report referenced during the investigation, with URL.

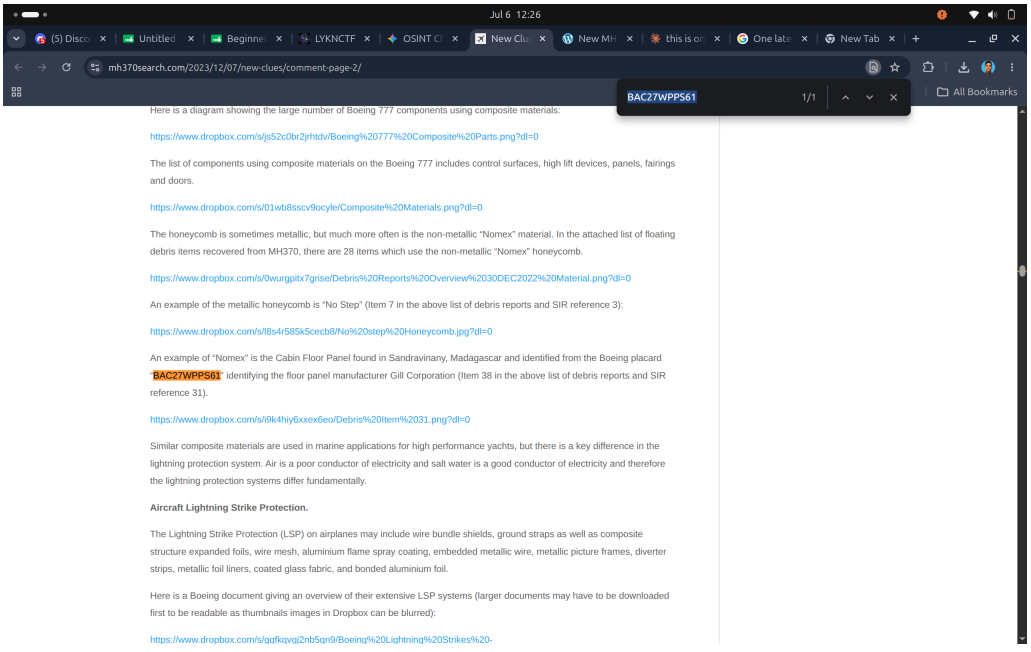


Figure 24: Detail of the marking linked to the Boeing cabin component.

Step 2. Formed all the credentials in the required order and got the flag.

- ITEM - ITEM31
- Marker - BAC27WPPS61
- Specification - BMS4-20

Flag: `LYKN{ITEM31_BAC27WPPS61_BMS4-20}`

Challenge 6 — Hash & Dash

Category

Cryptography

Problem Description

This is one of the cryptographic challenge, asking for login in into the server via the name of the admin and getting the hidden information that the admin is having. This is the classic JWT (”JSON Web Token”) forgery challenge.

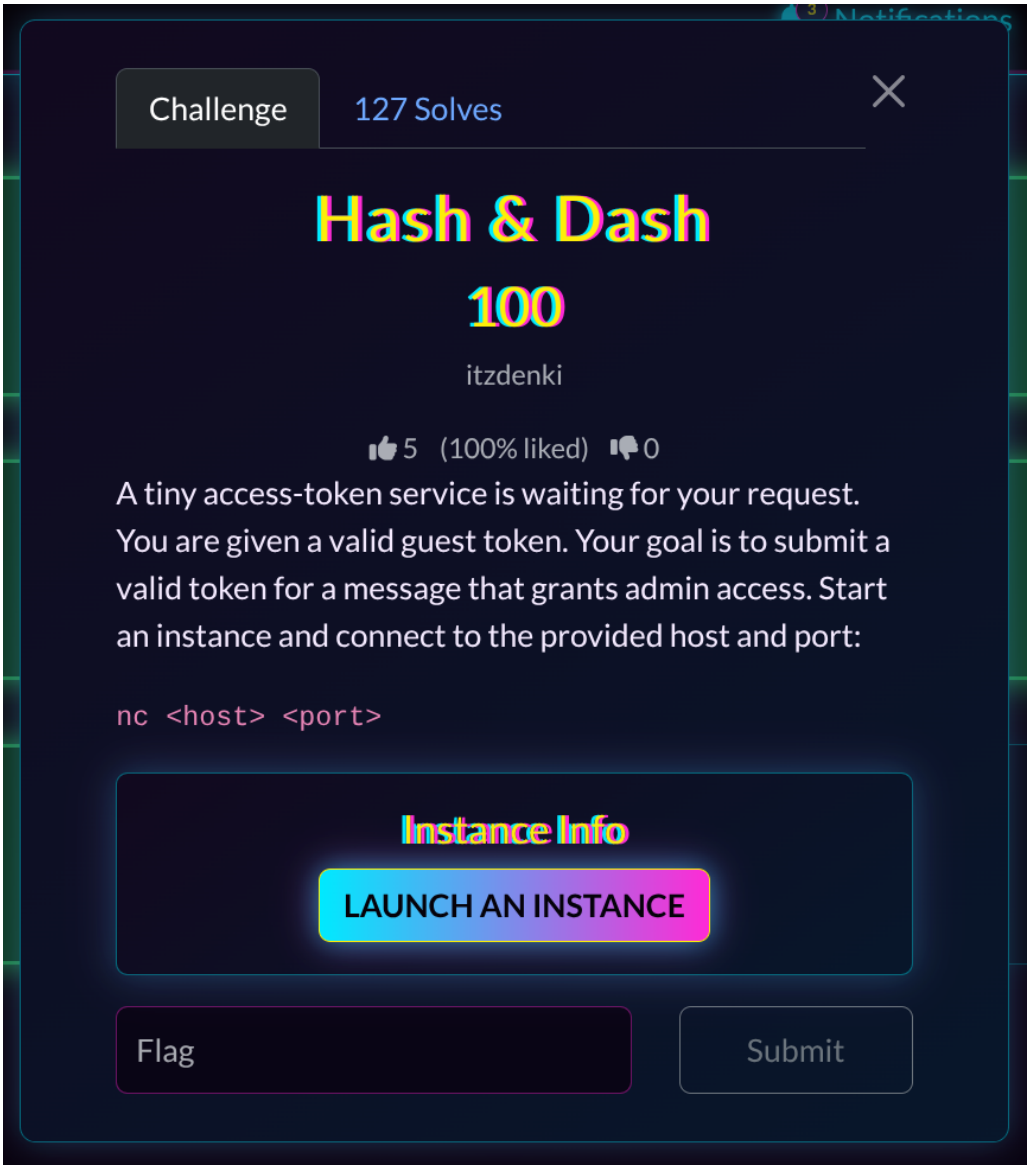


Figure 25: Challenge description for Hash & Dash.

The mentioned below are the vulnerability found in this challenge: -

1. The vulnerability: Hash Length-Extension Attack - The server authenticated messages via just concatenation of a secret key in front of the message and hashed the whole thing, then treating the final thing as the signature. Hence, in this we are required to find a correct hash for getting the access of admin.
2. The SHA-256 is a Merkle–Damgård construction. It doesn't hash our message in one sho: -
   - Pads our message with a `0x80` byte, some zero bytes, and an 8-byte length field (to a multiple of 64 bytes).
   - Splits it into 64-byte blocks.
   - Processes each block, updating an internal 256-bit state (8×32-bit words).

- The final thing we have is that raw internal state — nothing is hidden or mixed in afterward.

3. Hence, if I already know a valid SHA256(secret || message), I know the exact internal state of the hash function after it finished processing secret || message. I can resume hashing from that state and feed in more data — I never need to know secret itself, only its length (so I can correctly compute where the padding goes).

4. We have to determine the length of the secret, so that we can do the correct padding, which matches the server SHA – 256, 64 bit padding.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. With the help of LLM generated the code for the same vulnerability, and got the flag.

Mentioned below is the code that is being used: -

```
import socket, struct, json, re
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "51.79.140.18"
PORT = int(sys.argv[2]) if len(sys.argv) > 2 else 15191

# ---------- pure-python SHA-256 internals ----------
K = [
0x428a2f98,0x71374491,0xb5c0fbcf,0xe9b5dba5,0x3956c25b,0x59f111f1,0x923f82a4
    ,0xab1c5ed5,
0xd807aa98,0x12835b01,0x243185be,0x550c7dc3,0x72be5d74,0x80deb1fe,0x9bdc06a7
    ,0xc19bf174,
0xe49b69c1,0xefbe4786,0x0fc19dc6,0x240ca1cc,0x2de92c6f,0x4a7484aa,0x5cb0a9dc
    ,0x76f988da,
0x983e5152,0xa831c66d,0xb00327c8,0xbf597fc7,0xc6e00bf3,0xd5a79147,0x06ca6351
    ,0x14292967,
0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,0x650a7354,0x766a0abb,0x81c2c92e
    ,0x92722c85,
0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,0xd192e819,0xd6990624,0xf40e3585
    ,0x106aa070,
0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,0x391c0cb3,0x4ed8aa4a,0x5b9cca4f
    ,0x682e6ff3,
0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,0x90befffa,0xa4506ceb,0xbef9a3f7
    ,0xc67178f2]

def rrot(x, n):
    return ((x >> n) | (x << (32 - n))) & 0xffffffff

def sha256_pad(msg_len_bytes):
    ml_bits = msg_len_bytes * 8
    pad = b'\x80'
    pad_len = (56 - (msg_len_bytes + 1) % 64) % 64
    pad += b'\x00' * pad_len
```

```
    pad += struct.pack('>Q', ml_bits)
    return pad

def sha256_compress(chunk, h):
    w = list(struct.unpack('>16L', chunk)) + [0]*48
    for i in range(16, 64):
        s0 = rrot(w[i-15],7) ^ rrot(w[i-15],18) ^ (w[i-15] >> 3)
        s1 = rrot(w[i-2],17) ^ rrot(w[i-2],19) ^ (w[i-2] >> 10)
        w[i] = (w[i-16] + s0 + w[i-7] + s1) & 0xffffffff

    a,b,c,d,e,f,g,hh = h
    for i in range(64):
        S1 = rrot(e,6) ^ rrot(e,11) ^ rrot(e,25)
        ch = (e & f) ^ ((~e & 0xffffffff) & g)
        t1 = (hh + S1 + ch + K[i] + w[i]) & 0xffffffff
        S0 = rrot(a,2) ^ rrot(a,13) ^ rrot(a,22)
        maj = (a & b) ^ (a & c) ^ (b & c)
        t2 = (S0 + maj) & 0xffffffff
        hh, g, f = g, f, e
        e = (d + t1) & 0xffffffff
        d, c, b = c, b, a
        a = (t1 + t2) & 0xffffffff
    return [(x + y) & 0xffffffff for x, y in zip(h, [a,b,c,d,e,f,g,hh])]

def length_extend(orig_tag_hex, orig_total_len, extra):
    h = list(struct.unpack('>8L', bytes.fromhex(orig_tag_hex)))
    glue = sha256_pad(orig_total_len)
    new_len_before_extra = orig_total_len + len(glue)
    data = extra + sha256_pad(new_len_before_extra + len(extra))
    for i in range(0, len(data), 64):
        h = sha256_compress(data[i:i+64], h)
    new_tag = b''.join(struct.pack('>L', x) for x in h).hex()
    return new_tag, glue

# ---------- networking ----------
def recvline(sock):
    buf = b''
    while not buf.endswith(b'\n'):
        chunk = sock.recv(1)
        if not chunk:
            break
        buf += chunk
    return buf.decode(errors='replace').strip()

def recv_all_available(s, timeout=2.0):
    """Read whatever the server sends until it pauses (no newline required).
    """
    s.settimeout(timeout)
    chunks = []
    try:
        while True:
```

```
            chunk = s.recv(4096)
            if not chunk:
                break
            chunks.append(chunk)
            # heuristically stop if we've hit a prompt or got a full line
    already
            if chunk.endswith(b'> ') or chunk.endswith(b'\n'):
                # try a very short extra read in case more is coming right
    after
                s.settimeout(0.3)
                try:
                    more = s.recv(4096)
                    if more:
                        chunks.append(more)
                    else:
                        break
                except socket.timeout:
                    break
    except socket.timeout:
        pass
    return b''.join(chunks).decode(errors='replace')

def try_attack(secret_len, extra=b"&role=admin"):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5)
    s.connect((HOST, PORT))

    banner = recvline(s)  # the JSON line
    _instructions = recvline(s)  # "Submit one JSON line with msg and tag."
    # consume the trailing un-terminated "> " prompt (no newline, so
    recvline can't grab it)
    s.settimeout(0.5)
    try:
        s.recv(64)  # flushes "> "
    except socket.timeout:
        pass

    data = json.loads(banner)

    msg_hex = data["message_hex"]
    orig_tag = data["token"]
    orig_len = len(bytes.fromhex(msg_hex))

    total_len = secret_len + orig_len
    new_tag, glue = length_extend(orig_tag, total_len, extra)

    forged_msg_hex = msg_hex + glue.hex() + extra.hex()

    payload = json.dumps({"msg": forged_msg_hex, "tag": new_tag})
    s.sendall(payload.encode() + b"\n")
```

```
    resp = recv_all_available(s, timeout=2.0)
    s.close()
    return resp

def main():
    # Phase 1: discover secret length (skip if already known)
    KNOWN_SECRET_LEN = 16  # confirmed valid from prior run

    candidates = [
        b"&admin=true",
        b"&admin=1",
        b"&admin=True",
        b"&role=admin",
        b"&isadmin=true",
        b"&is_admin=true",
        b"&is_admin=1",
        b"&access=admin",
        b"&level=admin",
        b"&grant=admin",
        b"&privilege=admin",
        b"&user=admin",
        b"&user=admin&role=admin",
        b"&role=admin&admin=true",
        b"&role=administrator",
        b"&perm=admin",
        b"&permission=admin",
    ]

    for extra in candidates:
        resp = try_attack(KNOWN_SECRET_LEN, extra=extra)
        print(f"[extra={extra}] -> {resp.strip()[:200]}")
        if resp and '"admin": true' in resp.replace(" ", ""):
            print("\n[+] ADMIN GRANTED!")
            print(resp)
            return
        if resp and ("flag" in resp.lower() or re.search(r'[A-Za-z0-9_
    ]+\{.*?\}', resp)):
            print("\n[+] FLAG FOUND!")
            print(resp)
            return

    print("\n[-] None of the candidate fields worked. Need to inspect
    parsing logic further.")

if __name__ == "__main__":
    main()
```

`python3 solve.py 51.79.140.18 11240` - This is the command used for the running of the code.

Flag: `LYKNCTF{5c03b69c71014f67950b8296420d69cd}`

Challenge 7 — Route to Nowhere

Category

OSINT (Open Source Intelligence)

Problem Description

This is one of the OSINT challenge, in this challenge we are given the image of a the place in Hanoi, and we need to find the nearest bus — stop to this challenge.

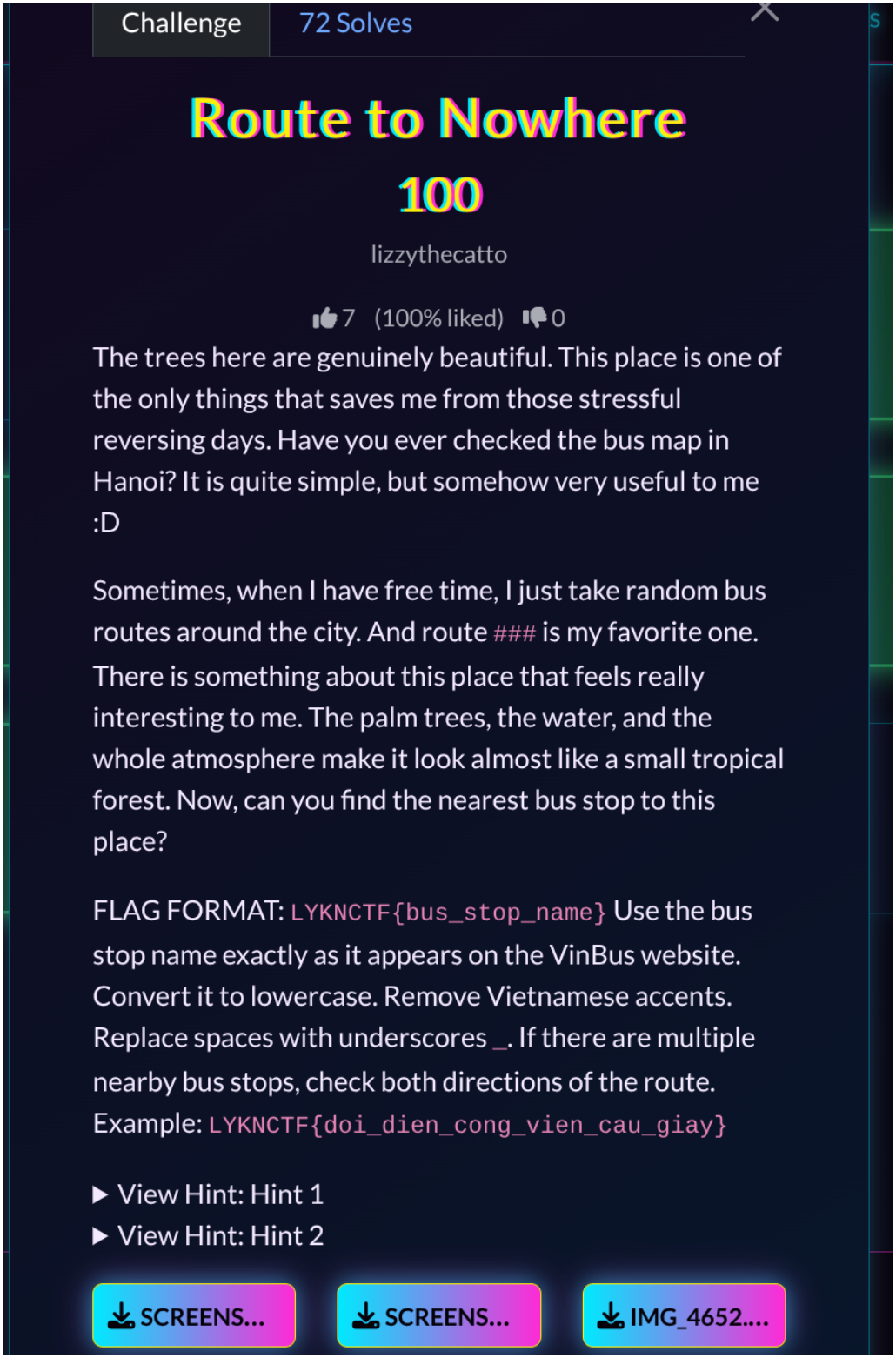


Figure 26: Challenge description for Route to Nowhere.

Figure 27: Attached image of the place in Hanoi.

Figure 28: Additional reference image for the challenge.

Figure 29: Additional reference image for the challenge.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Went to the website of VinBus website, found the route no. mentioned on the 1st image on the bus, went to this route no., and started looking for all the stops that are being mentioned in the same route.

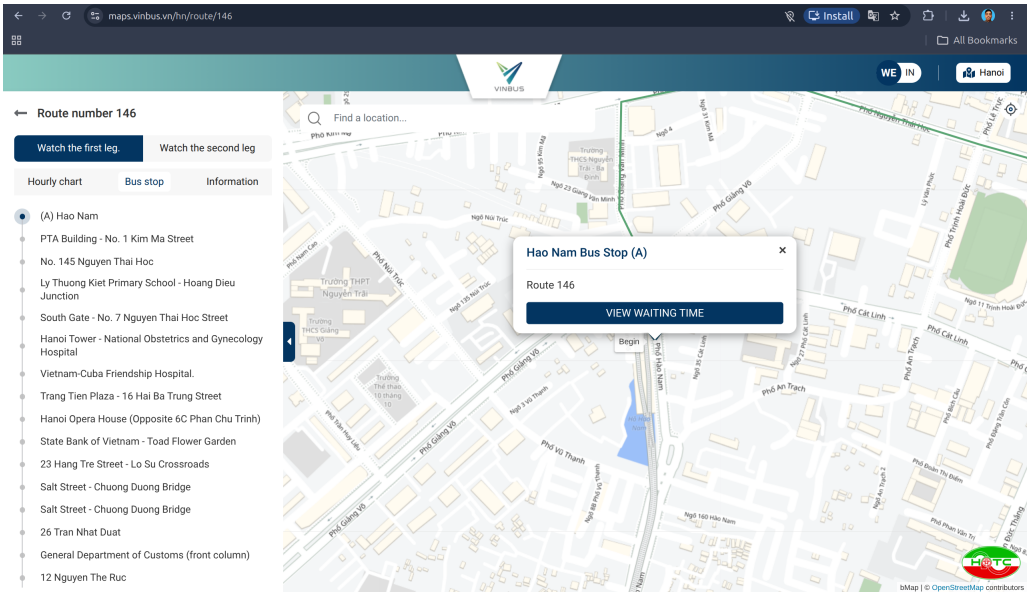


Figure 30: VinBus route lookup for the number identified on the bus.

Step 2. Used the name of the stops on Google Maps, and looked for the pond or lake nearby, found one the bus stop on the return route of Route — 146.

Step 3. The name of the bus stop is - Điếm Canh Đê Phù Dực

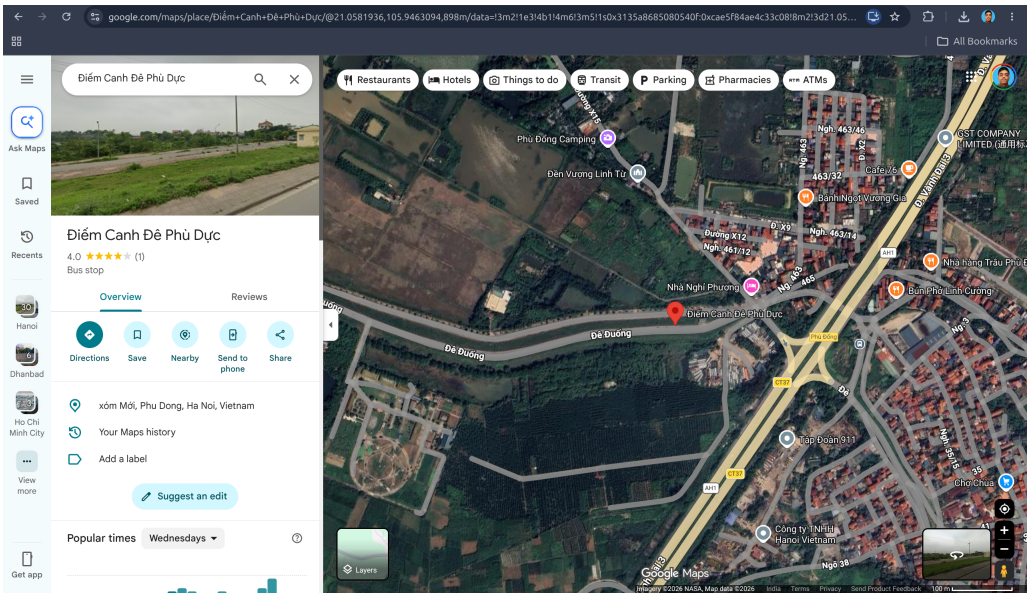


Figure 31: Google Maps view of the identified bus stop, Điếm Canh Đê Phù Dực.

Flag: `LYKNCTF{diem_canh_de_phu_duc}`

Challenge 8 — Waguri2

Category

Miscellaneous / Reverse Engineering

Problem Description

This is one of the reverse challenge, in this challenge we are given with some sort of language that is to be decrypted for finding something meaningful for finding the flag of the challenge.

Figure 32: Challenge description for Waguri2.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. The challenge is having the logic that is all related to the Brainfuck, logic and from there we are needed to the reverse engg. of the logic.

Step 2. The attachment uses character names from the manga Kaoru Hana

wa Rin to Saku (The Fragrant Flower Blooms With Dignity) as a direct substitution for Brainfuck commands. By analyzing the loops ([and]), pointer movements, and input positions, we can determine the exact translation table:

With the help of LLM found the exact translation table for the Brainfuck commands: -

Character Name	Brainfuck Symbol	Description
waguri_kaoruko	+	Increment the current cell
tsumugi_rintaro	-	Decrement the current cell
usami_shohei	>	Move the data pointer to the right
natsusawa_saku	<	Move the data pointer to the left
yorita_ayato	[	Jump forward if the current cell is 0
hoshina_subaru	]	Jump backward if the current cell is non-zero
kaoru_hana	,	Accept one byte of input

Figure 33: Translation table mapping manga character names to Brainfuck commands.

Step 3. With the help of LLM, wrote a code that is being used for the translation of all the word taken from manga, into the brainfuck code and then using the brainfuck interpreter for the execution of the same.

Mentioned below is the code that is being used: -

```
import sys

def translate_esolang(filename):
    # Mapping table from flower esolang tokens to standard
   Brainfuck symbols
    mapping = {
        "waguri_kaoruko": "+",
        "tsumugi_rintaro": "-",
        "usami_shohei": ">",
        "natsusawa_saku": "<",
        "yorita_ayato": "[",
        "hoshina_subaru": "]",
        "kaoru_hana": ","
    }
```

```
    try:
        with open(filename, "r") as f:
            tokens = f.read().split()
    except FileNotFoundError:
        print(f"Error: Could not find '{filename}' in this
    directory.")
        sys.exit(1)

    # Translate valid tokens, ignoring unexpected words
    bf_code = "".join(mapping[token] for token in tokens if token
    in mapping)
    return bf_code

def run_brainfuck(bf_code):
    print("\n--- Running Brainfuck Program ---")
    print("Type your input string below when prompted.")

    # Brainfuck interpreter implementation
    memory = [0] * 30000
    ptr = 0
    code_ptr = 0
    input_buffer = ""

    # Precompute bracket pairs for fast jumping
    bracket_map = {}
    stack = []
    for i, char in enumerate(bf_code):
        if char == "[":
            stack.append(i)
        elif char == "]":
            start = stack.pop()
            bracket_map[start] = i
            bracket_map[i] = start

    # Main execution loop
    while code_ptr < len(bf_code):
        cmd = bf_code[code_ptr]

        if cmd == ">":
            ptr += 1
        elif cmd == "<":
            ptr -= 1
        elif cmd == "+":
            memory[ptr] = (memory[ptr] + 1) % 256
        elif cmd == "-":
            memory[ptr] = (memory[ptr] - 1) % 256
        elif cmd == ".":
            sys.stdout.write(chr(memory[ptr]))
            sys.stdout.flush()
        elif cmd == ",":
            if not input_buffer:
```

```
                # Ask for user input if buffer is empty
                input_buffer = input("Input character(s): ") + "\n"
            if input_buffer:
                memory[ptr] = ord(input_buffer[0])
                input_buffer = input_buffer[1:]
        elif cmd == "[":
            if memory[ptr] == 0:
                code_ptr = bracket_map[code_ptr]
        elif cmd == "]":
            if memory[ptr] != 0:
                code_ptr = bracket_map[code_ptr]

        code_ptr += 1
    print("\n--- Program Finished Gracefully ---")

if __name__ == "__main__":
    # 1. Translate the file
    bf_program = translate_esolang("output.txt")
    print("Successfully translated to Brainfuck!")
    print(f"Brainfuck code snippet (first 100 chars): {bf_program
    [:100]}...")

    # 2. Save translated file for external analysis if needed
    with open("translated.bf", "w") as f:
        f.write(bf_program)
    print("Saved clean code to 'translated.bf'.")

    # 3. Interactively run the code
    run_brainfuck(bf_program)
```

Step 4. Got all the translation being into a file named as translated.bf. The code or the logic is asking us for an input, if our input is correct, it is going to input correct or false. Hence, if we give it a correct flag we will get a correct message and the flag.

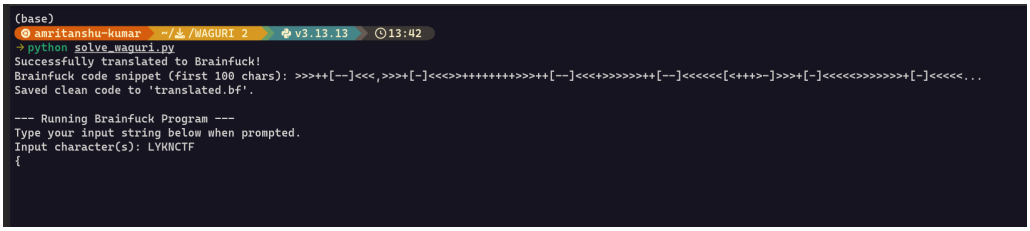


Figure 34: Running the translated Brainfuck program.

Step 5. With the help of LLM generated the code for brute-forcing the program and finding the correct flag. The challenge program is having a simple logic of either true or false, if we are giving a correct letter, it waits for the next letter, if we are giving a wrong letter it is getting stuck into an infinite loop.

Used the mentioned below program for brute — forcing: -

```
import sys

# 1. Load the clean Brainfuck code generated from your previous run
try:
    with open("translated.bf", "r") as f:
        bf_code = f.read().strip()
except FileNotFoundError:
    print("Error: Make sure 'translated.bf' is in this directory.")
    sys.exit(1)

# 2. Precompute bracket pairs for fast loop execution
bracket_map = {}
stack = []
for i, char in enumerate(bf_code):
    if char == "[":
        stack.append(i)
    elif char == "]":
        if stack:
            start = stack.pop()
            bracket_map[start] = i
            bracket_map[i] = start

def test_string(input_str):
    memory = [0] * 30000
    ptr = 0
    code_ptr = 0
    input_idx = 0
    instructions_run = 0
    max_instructions = 300000  # Cap execution to catch infinite loops on
    wrong answers

    while code_ptr < len(bf_code) and instructions_run < max_instructions:
        cmd = bf_code[code_ptr]
        instructions_run += 1

        if cmd == ">": ptr += 1
        elif cmd == "<": ptr -= 1
        elif cmd == "+": memory[ptr] = (memory[ptr] + 1) % 256
        elif cmd == "-": memory[ptr] = (memory[ptr] - 1) % 256
        elif cmd == ",":
            if input_idx < len(input_str):
                memory[ptr] = ord(input_str[input_idx])
                input_idx += 1
            else:
                # String is correct so far; it successfully reached the next
    prompt!
                return "WAITING"
        elif cmd == "[":
            if memory[ptr] == 0:
                code_ptr = bracket_map.get(code_ptr, code_ptr)
```

```
        elif cmd == "]":
            if memory[ptr] != 0:
                code_ptr = bracket_map.get(code_ptr, code_ptr)

        code_ptr += 1

        if code_ptr >= len(bf_code):
            return "FINISHED"
    return "FAILED"

# 3. Initialize with the known CTF flag prefix format
flag = "LYKNCTF{"
print(f"[+] Starting automated solve from: {flag}")

# Printable characters typical for CTF flags
charset = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_}"

while True:
    found_next = False
    for c in charset:
        res = test_string(flag + c)
        if res == "FINISHED":
            flag += c
            print(f"\n\n[SUCCESS] Flag found: {flag}")
            sys.exit(0)
        elif res == "WAITING":
            flag += c
            print(f"Discovering: {flag}", end="\r")
            sys.stdout.flush()
            found_next = True
            break

    if not found_next:
        print(f"\n[-] Stopped tracking. Last matched sequence: {flag}")
        break
```

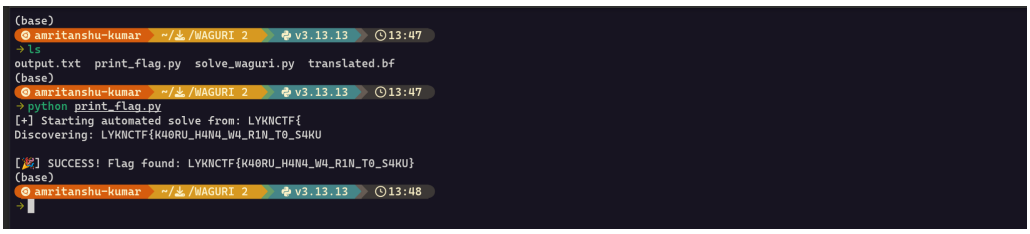


Figure 35: Final brute-force output revealing the flag.

Flag: `LYKNCTF{K4ORU_H4N4_W4_R1N_TO_S4KU}`

Challenge 9 — Inferior Student

Category

Reverse Engineering

Problem Description

This is one of the reverse challenge, here we are being given with a chall.exe and challl.py file, this is a flag check file, if we are entering a correct flag, then we are getting back correct message, else we are getting false.

But the issue is that all the things being mentioned in the code is in the form of the Greek language and it is being checking for the following mentioned things: -

1. Anti-debug checks that all XOR into one state variable.
2. That state variable becomes an AES-GCM key/IV. (AES (Advanced Encryption Standard) A symmetric-key block cipher — meaning the same secret key is used to both encrypt and decrypt.

   AES by itself only knows how to scramble a single 16-byte block. GCM is a mode of operation — a recipe for using AES repeatedly to encrypt data of any length, while also providing:

   - Confidentiality
   - Authentication)

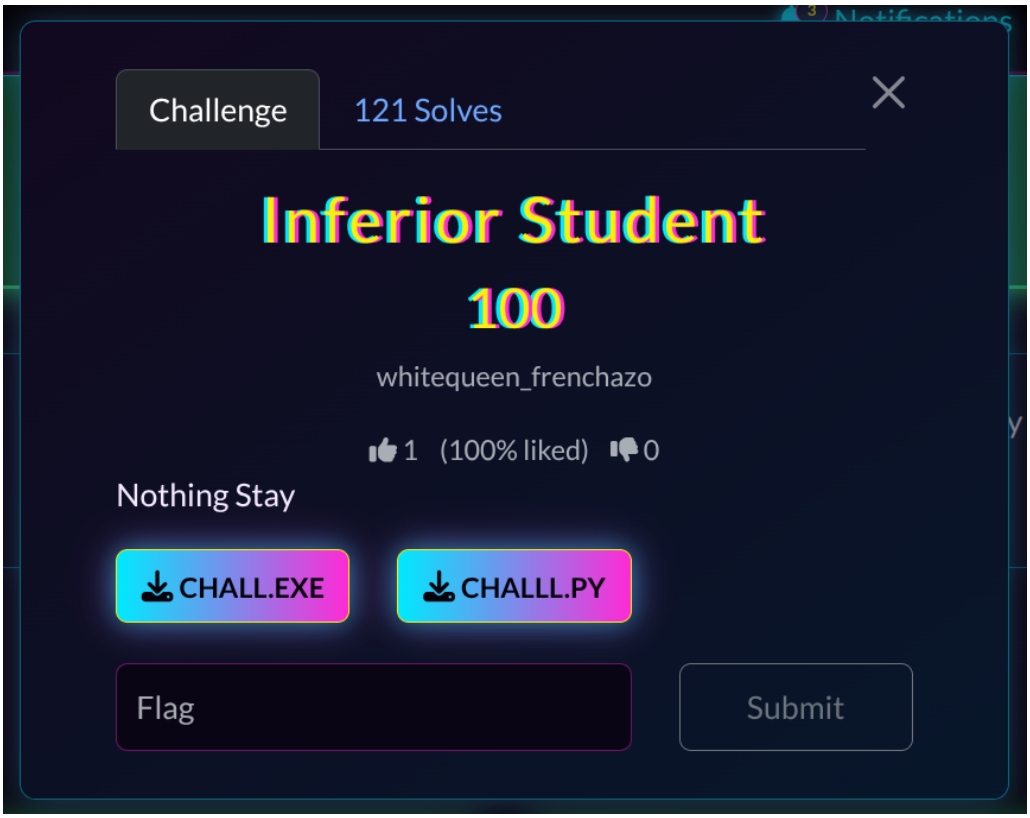
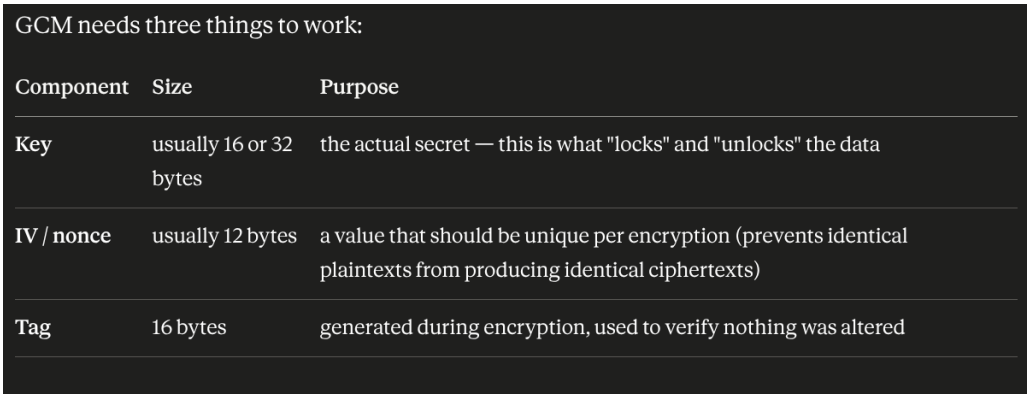


Figure 36: Challenge description for Inferior Student.



GCM needs three things to work:

Component	Size	Purpose
Key	usually 16 or 32 bytes	the actual secret — this is what "locks" and "unlocks" the data
IV / nonce	usually 12 bytes	a value that should be unique per encryption (prevents identical plaintexts from producing identical ciphertexts)
Tag	16 bytes	generated during encryption, used to verify nothing was altered

Figure 37: AES-GCM key/IV state derivation logic within the Greek-obfuscated code.

3. If the state is ”clean” (0), it correctly decrypts a marshaled Python code object, which then gets `exec()`’d.

I also found one of the command being called after the AES/GCM decryption setup. `exec(__f9b97b623dabbbc83fb3c__, ...)`

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. With the help of LLM generated the code for the finding of the flag, from this Greek language, .py file, and got the flag.

Used the mentioned below code for the solving of the challenge: -

```
#!/usr/bin/env python3
"""
Automated solver for the "Inferior Student" (LYKNCTF) style challenge.

Pipeline (all discovered manually first, automated here):

    challl.py (homoglyph-obfuscated, anti-debug)
      | exec(<code object>, globals)  <- we dump this code object
      v
    stage2 code object (also homoglyph-obfuscated)
      | AES-GCM decrypts one or more blobs
      v
    a blob that starts with the zlib magic byte (0x78)
      | zlib.decompress -> marshal.loads
      v
    stage3 code object: builds a ChaCha20 cipher from a fixed key/nonce,
    encrypts input(), compares to a stored ciphertext constant.

Since ChaCha20 is a symmetric stream cipher, once we recover (key, nonce,
ciphertext) we don't need the correct input at all -- we just run the
cipher on the stored ciphertext to get the plaintext flag back.

Design notes on *why* no de-obfuscation of identifiers was needed:
  - Stage 1 -> Stage 2: we never read stage 1's logic. We locate the
```

```
    single `exec(NAME, ...)` call textually and splice in one line that
    dumps NAME via marshal *before* it runs. This changes no runtime
    behavior, so none of the anti-debug/anti-tamper checks (which key off
    things like hasattr(builtins.compile, '__code__')) are disturbed.
  - Stage 2 -> Stage 3: instead of reading 5000+ garbled names, we
    monkeypatch
    the *library* entry points the obfuscated code must eventually call
    (hashlib.sha256, Crypto.Cipher.AES.new, cryptography's Cipher.decryptor,
    and ChaCha20 construction) and let stage 2 run normally. We watch data
    flow through real crypto primitives instead of reading variable names.
  - Stage 3: small enough to read directly, but we still don't rename
    anything -- we identify key/nonce/ciphertext by *how* they're used
    (passed into ChaCha20(...), compared with ==) rather than by name.

Usage:
    python3 solve_inferior_student.py /path/to/challl.py
"""

import sys
import os
import re
import io
import zlib
import marshal
import types
import hashlib
import subprocess
import tempfile

FLAG_RE = re.compile(rb'[A-Za-z0-9_]{2,20}\{[ -~]{4,300}\}')

#
    ----------------------------------------------------------------------------

# Stage 1: splice a dump statement in front of the final exec(), run it once
    ,
# and load the code object it would otherwise have exec()'d.
#
    ----------------------------------------------------------------------------

def extract_stage2_code_object(src_path):
    data = open(src_path, 'rb').read()

    # Find bare `exec(` calls (not `.exec(` / attribute access) at module
    scope.
    candidates = [m.start() for m in re.finditer(rb'(?<![.\w])exec\(', data)
    ]
    if not candidates:
        raise RuntimeError("No exec( call found in source -- structure
    differs from expected.")
```

```
    # The loader call is the *last* exec( in the file in this challenge
    family.
    idx = candidates[-1]
    line_start = data.rfind(b'\n', 0, idx) + 1
    indent = data[line_start:idx]
    indent = indent[: len(indent) - len(indent.lstrip(b' \t'))]

    m = re.match(rb'exec\(([^\s,()]+)\s*,', data[idx:idx + 300])
    if not m:
        raise RuntimeError("Could not parse the code-object argument of exec
    (...).")
    varname = m.group(1)

    with tempfile.NamedTemporaryFile(delete=False, suffix='.marshal') as tf:
        dump_path = tf.name

    dump_stmt = (
        indent + b'__builtins__.open(' + repr(dump_path).encode() +
        b',"wb").write(__import__("marshal").dumps(' + varname + b'))\r\n'
    )
    patched = data[:line_start] + dump_stmt + data[line_start:]

    with tempfile.NamedTemporaryFile(delete=False, suffix='.py') as tf:
        patched_path = tf.name
        tf.write(patched)

    # Run it once, unmodified environment, feeding harmless input.
    # We don't care about the result -- we only want the dumped code object.
    subprocess.run(
        [sys.executable, patched_path],
        input=b"dummy_input_value\n",
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
        timeout=60,
    )
    os.unlink(patched_path)

    if not os.path.exists(dump_path) or os.path.getsize(dump_path) == 0:
        raise RuntimeError("Dump file was not produced -- exec target may
    not be a code object, "
                           "or anti-debug checks altered behavior in this
    environment.")

    with open(dump_path, 'rb') as f:
        code = marshal.load(f)
    os.unlink(dump_path)
    return code

#
    ------------------------------------------------------------------------
```

```
# Stage 2+: monkeypatch crypto primitives, execute the code object, and
# recursively unwrap any AES-decrypted zlib+marshal payloads we find.
# Also capture every ChaCha20(key, nonce) construction we see.
#
    ------------------------------------------------------------------------

class Captured:
    def __init__(self):
        self.aes_plaintexts = []     # bytes blobs decrypted via AES
        self.chacha_params = []      # (key, nonce) tuples
        self.other_plaintexts = []   # bytes blobs decrypted via generic `
    cryptography` Cipher

def run_with_crypto_taps(code_obj, captured: Captured, feed_input=b"
    dummy_input_value"):
    """exec() a code object with real crypto libraries wrapped so every
    decrypt call and every ChaCha20 construction is logged, without changing
    what the code actually computes. All patches are restored afterwards
    (try/finally) so nested/recursive calls never see stale patches."""
    restore_fns = []  # list of (obj, attr, original_value) to restore in
    finally

    # --- pycryptodome AES ---
    try:
        from Crypto.Cipher import AES as _AES
        real_aes_new = _AES.new
        restore_fns.append((_AES, 'new', real_aes_new))

        def tapped_aes_new(key, mode, *a, **kw):
            cipher = real_aes_new(key, mode, *a, **kw)
            real_decrypt = cipher.decrypt

            def tapped_decrypt(data):
                pt = real_decrypt(data)
                captured.aes_plaintexts.append(pt)
                return pt

            cipher.decrypt = tapped_decrypt
            return cipher

        _AES.new = tapped_aes_new
    except ImportError:
        pass

    # --- `cryptography` hazmat Cipher (used for both AES-GCM/CTR and
    ChaCha20
    # in this challenge family; we tap the constructor and decryptor().
    update).
    # NOTE: the returned CipherContext is a Rust object with no __dict__, so
    # we can NOT assign .update on it -- we wrap it in a thin Python proxy
    # instead of mutating the object itself. ---
```

```
    try:
        from cryptography.hazmat.primitives.ciphers import Cipher as _Cipher
    , algorithms as _algorithms
        real_chacha20_init = _algorithms.ChaCha20.__init__
        restore_fns.append((_algorithms.ChaCha20, '__init__',
    real_chacha20_init))

        def tapped_chacha20_init(self, key, nonce, *a, **kw):
            captured.chacha_params.append((bytes(key), bytes(nonce)))
            return real_chacha20_init(self, key, nonce, *a, **kw)

        _algorithms.ChaCha20.__init__ = tapped_chacha20_init

        real_decryptor = _Cipher.decryptor
        restore_fns.append((_Cipher, 'decryptor', real_decryptor))

        def tapped_decryptor(self):
            d = real_decryptor(self)
            real_update = d.update

            class _Wrapped:
                def update(inner_self, data):
                    pt = real_update(data)
                    captured.other_plaintexts.append(pt)
                    return pt

                def __getattr__(inner_self, name):
                    return getattr(d, name)

            return _Wrapped()

        _Cipher.decryptor = tapped_decryptor
    except ImportError:
        pass

    # Feed a dummy answer to any input() calls inside this stage.
    def tapped_input(*a, **kw):
        return feed_input.decode(errors='replace')

    orig_builtin_input = getattr(__builtins__, 'input', None)
    try:
        __builtins__.input = tapped_input
    except AttributeError:
        orig_builtin_input = None

    g = {'__name__': '__main__', '__builtins__': __builtins__}
    try:
        exec(code_obj, g)
    except SystemExit:
        pass
    except Exception:
```

```
        pass  # we only care about side-effect captures, not the checker's
    own output
    finally:
        for obj, attr, orig in restore_fns:
            try:
                setattr(obj, attr, orig)
            except Exception:
                pass
        if orig_builtin_input is not None:
            try:
                __builtins__.input = orig_builtin_input
            except Exception:
                pass

def find_flag_in_bytes_pool(pools, chacha_params):
    """Try decrypting every candidate ciphertext-looking bytes blob with
    every
    captured ChaCha20 (key, nonce) pair, and check for a FLAG{...}-shaped
    result."""
    from cryptography.hazmat.primitives.ciphers import Cipher, algorithms

    candidates = []
    for blob in pools:
        if isinstance(blob, (bytes, bytearray)) and 8 <= len(blob) <= 4096:
            candidates.append(bytes(blob))

    for key, nonce in chacha_params:
        for ct in candidates:
            try:
                c = Cipher(algorithms.ChaCha20(key, nonce), mode=None)
                pt = c.decryptor().update(ct)
            except Exception:
                continue
            m = FLAG_RE.search(pt)
            if m:
                return m.group(0), key, nonce, ct
    return None

def walk_code_consts_for_bytes(code_obj, acc):
    """Collect every bytes-like constant reachable from a code object,
    including nested lambda/genexpr code objects, as fallback candidates.
    Important: this obfuscator frequently does NOT store raw bytes constants
    in the bytecode. Instead it stores a tuple of ints and reconstructs the
    bytes object at runtime via `bytes([169, 136, 202, ...])`. So we also
    treat any plausible int-tuple (all values 0-255, reasonable length) as a
    candidate byte blob, not just literal bytes/bytearray constants."""
    for c in code_obj.co_consts:
        if isinstance(c, (bytes, bytearray)):
            acc.append(bytes(c))
        elif isinstance(c, types.CodeType):
            walk_code_consts_for_bytes(c, acc)
```

```
        elif isinstance(c, tuple) and 4 <= len(c) <= 4096:
            if all(isinstance(x, int) and 0 <= x <= 255 for x in c):
                acc.append(bytes(c))

def main():
    if len(sys.argv) != 2:
        print(f"Usage: {sys.argv[0]} <path-to-challl.py>")
        sys.exit(1)

    src_path = sys.argv[1]

    print("[*] Stage 1: locating final exec() and dumping the decrypted code
     object...")
    stage2_code = extract_stage2_code_object(src_path)
    print(f"    -> dumped stage-2 code object ({len(stage2_code.co_names)}
    names, "
          f"{len(stage2_code.co_consts)} consts)")

    captured = Captured()
    print("[*] Stage 2: executing with crypto taps installed (AES / ChaCha20
     / Cipher)...")
    run_with_crypto_taps(stage2_code, captured)

    print(f"    -> captured {len(captured.aes_plaintexts)} AES plaintext
    blob(s), "
          f"{len(captured.chacha_params)} ChaCha20 key/nonce pair(s) so far"
    )

    # Peel any zlib+marshal code objects out of what AES gave us,
    recursively.
    pending = list(captured.aes_plaintexts)
    all_bytes_pool = list(captured.aes_plaintexts)
    depth = 0
    while pending:
        depth += 1
        blob = pending.pop(0)
        if len(blob) > 2 and blob[0] == 0x78:  # zlib header
            try:
                dec = zlib.decompress(blob)
            except Exception:
                continue
            try:
                obj = marshal.loads(dec)
            except Exception:
                continue
            if isinstance(obj, types.CodeType):
                print(f"[*] Stage {depth + 2}: found nested code object
    under zlib -- executing with taps...")
                walk_code_consts_for_bytes(obj, all_bytes_pool)
                sub_captured = Captured()
                run_with_crypto_taps(obj, sub_captured)
```

```
                captured.chacha_params.extend(sub_captured.chacha_params)
                all_bytes_pool.extend(sub_captured.aes_plaintexts)
                all_bytes_pool.extend(sub_captured.other_plaintexts)
                pending.extend(sub_captured.aes_plaintexts)

    print(f"[*] Total ChaCha20 key/nonce pairs captured: {len(captured.
    chacha_params)}")
    print(f"[*] Total candidate ciphertext blobs to try: {len(all_bytes_pool
    )}")

    print("[*] Attempting to recover flag via direct ChaCha20 decryption "
          "(no brute force -- stream cipher is symmetric)...")
    result = find_flag_in_bytes_pool(all_bytes_pool, captured.chacha_params)

    if result is None:
        print("[!] Could not automatically recover the flag. Dumping
    captured "
              "material for manual inspection:")
        for i, (k, n) in enumerate(captured.chacha_params):
            print(f"    key[{i}]  = {k.hex()}")
            print(f"    nonce[{i}]= {n.hex()}")
        sys.exit(2)

    flag, key, nonce, ct = result
    print()
    print("=" * 60)
    print(f"FLAG: {flag.decode()}")
    print("=" * 60)
    print(f"  key   = {key.hex()}")
    print(f"  nonce = {nonce.hex()}")
    print(f"  ct    = {ct.hex()}")

    # Final sanity check: feed it into the *original*, unmodified script.
    print()
    print("[*] Verifying against the original, unmodified script...")
    proc = subprocess.run(
        [sys.executable, src_path],
        input=flag.decode().encode() + b"\n",
        capture_output=True,
        timeout=60,
    )
    out = proc.stdout.decode(errors='replace')
    print(f"    original script output: {out.strip()!r}")
    if 'Wrong' in out:
        print("[!] WARNING: original script rejected this flag. Investigate
    further.")
    else:
        print("[+] Flag verified against the original script.")

if __name__ == '__main__':
    main()
```

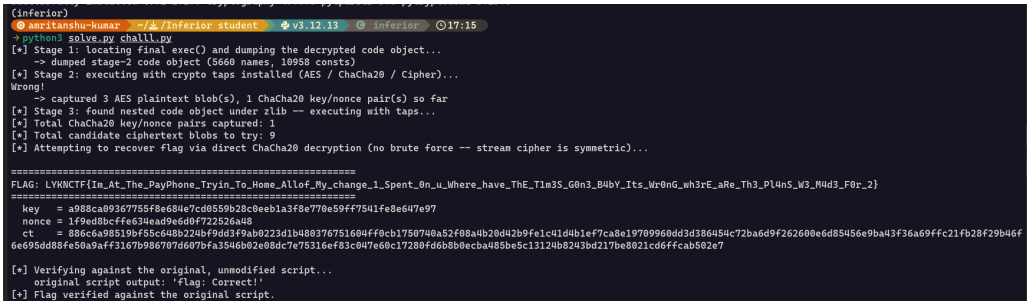


Figure 38: Final solver output confirming the recovered flag.

Flag: `LYKNCTF{Im_At_The_PayPhone_Tryin_To_Home_Allof_My_change_1_Spent_0n_u_Where_have_ThE_T1m3S_G0n3_B4bY_Its_Wr0nG_wh3rE_aRe_Th3_Pl4nS_W3_M4d3_F0r_2}`

Challenge 10 — Far Away

Category

OSINT (Open Source Intelligence)

Problem Description

This is one of the basic OSINT challenge for the finding of the name of the mountain, that is close to the college, whose image is being mentioned to us: -

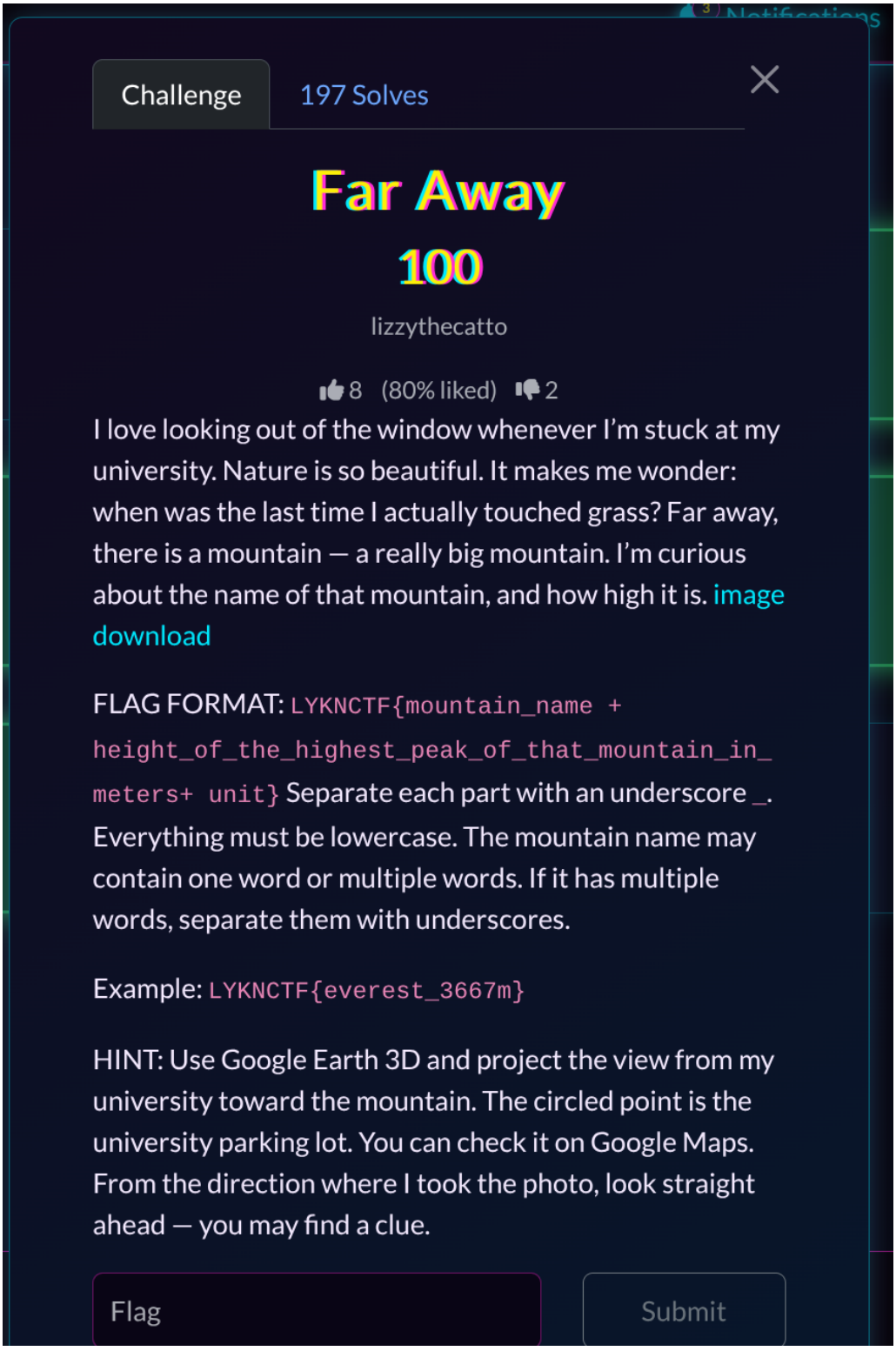


Figure 39: Challenge description for Far Away.

https://drive.google.com/drive/folders/1zJ5tO_xUID-XnbLFJ5zbi6fe6SnHt17I — The link for the image that is being mentioned.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Uploaded the image on GOOGLE IMAGE and found the name of the college and the name of the mountain along with the height. Hence, got the flag.

Figure 40: Google Image search result identifying the college and the nearby mountain.

Flag: `LYKNCTF{ba_vi_1296m}`

Challenge 11 — Far Away Revenge

Category

OSINT (Open Source Intelligence)

Problem Description

This is the continuation of the previous challenge, but this time we are required to give the details about the college, that how is the image is being captured from the hostel.

Figure 41: Challenge description for Far Away Revenge.

https://drive.google.com/drive/folders/1zJ5tO_xUID-XnbLFJ5zbi6fe6SnHt17I — The link for the image that is being mentioned.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. Uploaded the image on GOOGLE IMAGE, and GOOGLE GEMINI, found the details and the various permutations for the flag, hence got the flag.

Figure 42: Google Image / Google Gemini result used to identify the building details.

Step 2. The following are the credentials required for the flag: -

- Building name - delta
- Floor Number — 4
- Longitude - 21.01
- Latitude - 105.52

Flag: `LYKNCTF{delta_floor_4_21.01_105.52}`

Challenge 12 — Sleepless Machine

Category

Cryptography

Problem Description

This is one of the Cryptographic challenge, the challenge was having some sort of vulnerability with the vectors and the dimensions of a lattice.

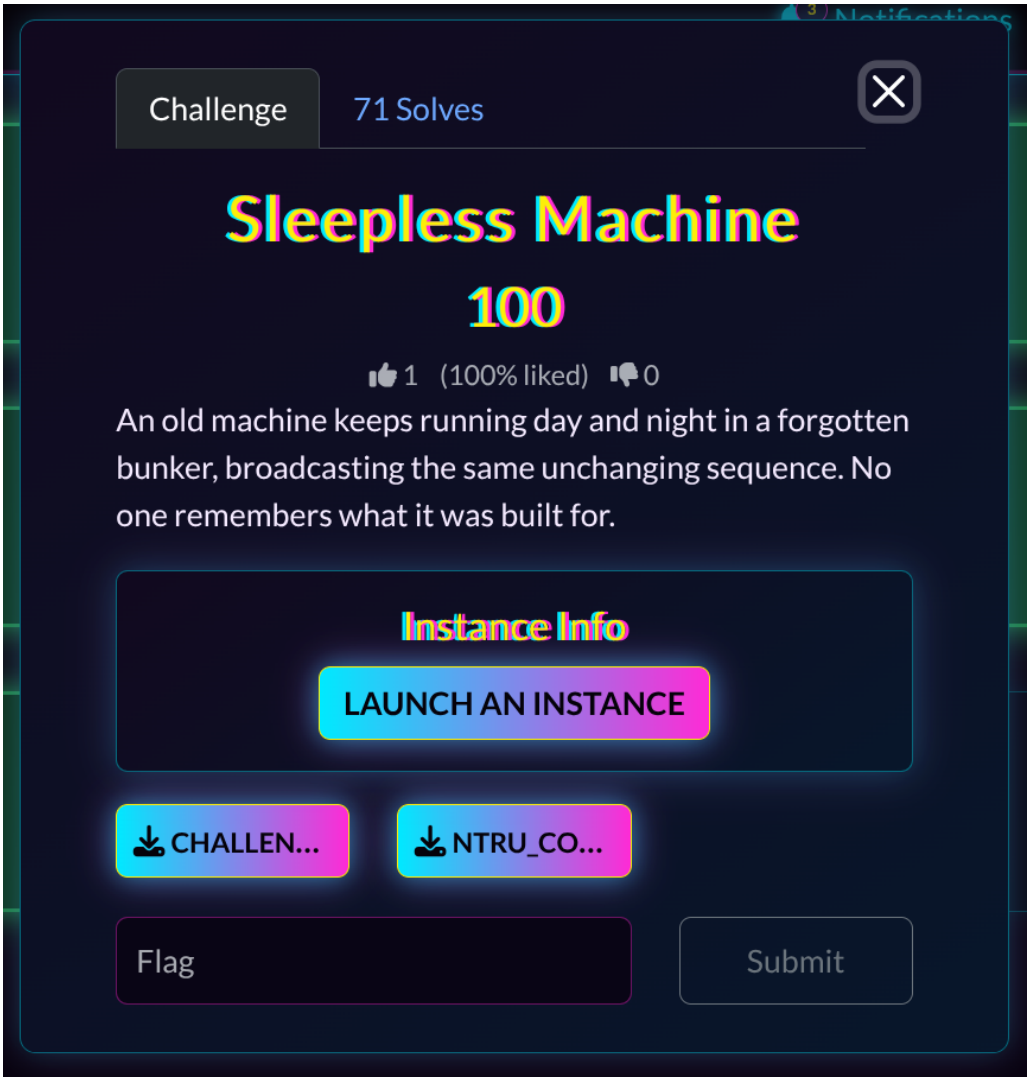


Figure 43: Challenge description for Sleepless Machine.

As I said I solved this challenge in a lazy — manner, hence I solved this challenge with the help of LLM. The challenge is consist of some tough mathematics and vulnerability.

The theory for the challenge is mentioned below: -

1. The Extremely Short Vector (The LLL Magnet)

When the server generated your public key $h(x) \equiv g(x) \cdot f^{-1}(x) \pmod{x^N - 1, Q}$, it used polynomials where coefficients were strictly ternary ($\{-1, 0, 1\}$) with a very low density of non-zero terms (36%).

If you look at the Euclidean length (norm) of a typical random vector in this 254-dimensional lattice modulo $Q = 4093$, its length is massive:

$$\text{Average Vector Length} \approx \sqrt{N} \cdot Q \approx \sqrt{127} \cdot 4093 \approx 46{,}125$$

However, your secret vector (f, g) only contains tiny $\{-1, 0, 1\}$ numbers. Its Euclidean length is:

$$\text{Secret Vector Length} = \sqrt{\sum f_i^2 + \sum g_i^2} \approx \sqrt{2 \cdot 127 \cdot 0.36} \approx 9.5$$

Your secret vector is $\sim 4{,}800$ times shorter than the surrounding math! Because of this massive gap, lattice reduction algorithms like LLL and BKZ don't even have to work hard—they act like a powerful magnet, snapping directly onto this ultra-short vector in just a few seconds.

2. The Hall of Mirrors (Why Raw LLL Wasn't Enough)

If LLL finds the short vector so easily, why did we need progressive BKZ and precision tweaks? Because in a cyclic polynomial ring modulo $x^N - 1$, any cyclic shift (rotation) of the secret polynomial has the exact same Euclidean length:

$$\|f(x)\| = \|x \cdot f(x)\| = \|x^2 \cdot f(x)\| = \cdots = \|x^{126} \cdot f(x)\|$$

When LLL reduces the lattice, it lands in a "hall of mirrors." It will hand you one of the 127 possible cyclic rotations of $f(x)$ (or its negative orientation $-f(x)$), but it does not know which specific rotation the server originally generated.

Because the challenge derived the AES key using weighted_trace—a function that multiplies each coefficient by its index $(i + 1)$—the key derivation is not shift-invariant. If you use the wrong rotation from LLL's output, your AES decryption tag check fails!

3. The GPS Anchor (The Even/Odd Parity Leakage)

This is where the author's intentional "leakage" (s_{even} and s_{odd}) defeated the hall of mirrors.

Because $N = 127$ is an odd number, shifting a polynomial cyclically by 1 position swaps the indices of all coefficients:

- What was at index 0 (even) moves to index 1 (odd).

- What was at index 1 (odd) moves to index 2 (even).
- What was at index 126 (even) wraps around to index 0 (even).

This means every single one of the 127 rotations produces a unique, predictable fingerprint of even-indexed and odd-indexed coefficient sums. By checking the LLL candidate vector against the server's leaked sums, your script acted like a GPS: it instantly filtered out the 126 "mirage" rotations, locked onto the exact alignment the server used, and derived the correct AES key.

Approach and Solution

Used the mentioned below commands for the retrieval of the flag: -

Step 1. First ran the test on the local machine for the checking of the correct execution of the logic, generated the code with the help of LLM. After successful recovery of the test code, ran it on the actual server.

Step 2. Ran the code on the actual server for getting the flag. Generated the code with the help of LLM.

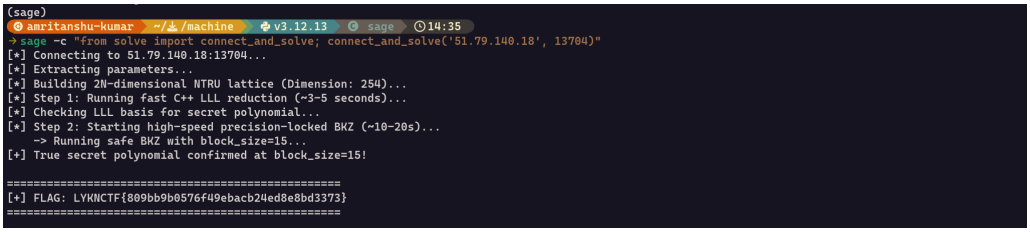


Figure 44: Running the solver against the live server.

Used the mentioned below code for the generation of the flag: -

```
import json
import socket
import sys
from random import Random as PyRandom
from Crypto.Cipher import AES
from challenge_gen import _derive_key, generate_instance
from ntru_core import sum_even_odd, weighted_trace
from sage.all import *

def safe_bkz(M, bs):
    """Runs BKZ using extended precision ('qd', 'dd', 'ld') to prevent Babai
    infinite loops. Never falls back to dangerous 53-bit float math!
    """
    # 1. Try Sage's wrapper with non-hyphenated ('qd', 'dd') and hyphenated
    names
    for kw in ["fp", "float_type"]:
        for val in ["qd", "dd", "ld", "long double", "mpfr", "q-d", "d-d"]:
            try:
                return M.BKZ(block_size=bs, **{kw: val})
```

```
            except Exception:
                continue

    # 2. Try calling fpylll directly if Sage wrapper strings fail
    try:
        from fpylll import BKZ as fpylll_BKZ, BKZParam, IntegerMatrix
        A = IntegerMatrix.from_matrix(M)
        for val in ["qd", "dd", "ld", "long double", "q-d", "d-d"]:
            try:
                param = BKZParam(block_size=bs, float_type=val)
                fpylll_BKZ.reduction(A, param)
                return A.to_matrix(Matrix(ZZ, M.nrows(), M.ncols()))
            except Exception:
                continue
    except Exception:
        pass

    # 3. SAFETY NET: NEVER call plain 53-bit M.BKZ() as it causes Babai
    crashes!
    # Instead, use exact NTL deep LLL (delta=0.9999), which achieves BKZ
    quality stably.
    print(
        " [!] High-precision BKZ flags unavailable. Using exact NTL Deep"
        " LLL...",
        flush=True,
    )
    try:
        return M.LLL(delta=0.9999, algorithm="NTL:LLL")
    except Exception:
        return M.LLL(delta=0.9999)

def check_basis(M, se_f, so_f, se_g, so_g, N, Q, Q_PRIME, enc):
    """Scans the lattice basis, tests cyclic shifts, and verifies via AES
    decryption."""
    nonce = bytes.fromhex(enc["nonce"])
    ciphertext = bytes.fromhex(enc["ciphertext"])
    tag = bytes.fromhex(enc["tag"])

    for row in M:
        f_cand = list(row[:N])
        g_cand = list(row[N:])
        if all(c == 0 for c in f_cand):
            continue

        for sign in [1, -1]:
            f_try = [sign * x for x in f_cand]
            g_try = [sign * x for x in g_cand]

            for shift in range(N):
                f_rot = [f_try[(i - shift) % N] for i in range(N)]
                g_rot = [g_try[(i - shift) % N] for i in range(N)]
```

```
                if (
                    sum_even_odd(f_rot, N) == (se_f, so_f)
                    and sum_even_odd(g_rot, N) == (se_g, so_g)
                ):
                    s_alg = weighted_trace(f_rot, g_rot, N, Q_PRIME)
                    key = _derive_key(s_alg, N, Q, Q_PRIME)
                    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
                    try:
                        flag = cipher.decrypt_and_verify(ciphertext, tag)
                        return f_rot, g_rot, flag
                    except Exception:
                        continue
    return None, None, None

def solve_instance(public_instance):
    print("[*] Extracting parameters...", flush=True)
    params = public_instance["parameters"]
    N = params["N"]
    Q = params["q"]
    Q_PRIME = params["q_prime"]
    h = public_instance["public_key"]["h"]
    leakage = public_instance["leakage"]
    enc = public_instance["encrypted_flag"]

    se_f, so_f = leakage["f_even_sum"], leakage["f_odd_sum"]
    se_g, so_g = leakage["g_even_sum"], leakage["g_odd_sum"]

    print(
        f"[*] Building 2N-dimensional NTRU lattice (Dimension: {2*N})...",
        flush=True,
    )
    M = Matrix(ZZ, 2 * N, 2 * N)
    for i in range(N):
        M[i, i] = 1
        for j in range(N):
            M[i, N + ((i + j) % N)] = h[j]
    for i in range(N):
        M[N + i, N + i] = Q

    # Step 1: Fast C++ LLL reduction (~3 seconds)
    print(
        "[*] Step 1: Running fast C++ LLL reduction (~3-5 seconds)...",
        flush=True,
    )
    M = M.LLL(delta=0.99)

    print("[*] Checking LLL basis for secret polynomial...", flush=True)
    f_secret, g_secret, flag = check_basis(
        M, se_f, so_f, se_g, so_g, N, Q, Q_PRIME, enc
    )
```

```
    # Step 2: Progressive BKZ with precision-locked safe_bkz (100% crash-
    proof)
    if flag is None:
        print(
            "[*] Step 2: Starting high-speed precision-locked BKZ"
            " (~10-20s)...",
            flush=True,
        )
        for bs in [15, 20, 25, 30]:
            print(
                f" -> Running safe BKZ with block_size={bs}...", flush=True
            )
            M = safe_bkz(M, bs)
            f_secret, g_secret, flag = check_basis(
                M, se_f, so_f, se_g, so_g, N, Q, Q_PRIME, enc
            )
            if flag is not None:
                print(
                    f"[+] True secret polynomial confirmed at block_size={bs
    }!",
                    flush=True,
                )
                break

    if flag is None:
        print(
            "[-] Failed to find verified target polynomial in BKZ basis.",
            flush=True,
        )
        return

    print("\n" + "=" * 50, flush=True)
    print(f"[+] FLAG: {flag.decode()}", flush=True)
    print("=" * 50 + "\n", flush=True)

def test_local():
    print("[*] Running LOCAL verification test...", flush=True)
    rng = PyRandom(1337)
    test_flag = "LYKNCTF{test_local_lattice_reduction_success}"
    inst, _ = generate_instance(test_flag, rng)
    solve_instance(inst)

def connect_and_solve(host, port):
    print(f"[*] Connecting to {host}:{port}...", flush=True)
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.settimeout(15)
            s.connect((host, port))
            data = ""
            while True:
```

```
                try:
                    chunk = s.recv(4096).decode("utf-8", errors="ignore")
                    if not chunk:
                        break
                    data += chunk
                    if "{" in data and "}" in data:
                        try:
                            json_start = data.find("{")
                            json_end = data.rfind("}") + 1
                            instance_data = json.loads(
                                data[json_start:json_end]
                            )
                            break
                        except Exception:
                            continue
                except socket.timeout:
                    break

            if not data.strip():
                print(
                    "[-] Received empty response. Is the CTF instance
    expired?",
                    flush=True,
                )
                return

            json_start = data.find("{")
            json_end = data.rfind("}") + 1
            if json_start == -1 or json_end == 0:
                print("[-] No valid JSON found in server response:", flush=
    True)
                print(data, flush=True)
                return

            instance_data = json.loads(data[json_start:json_end])
            solve_instance(instance_data)
    except Exception as e:
        print(f"[-] Connection or parsing error: {e}", flush=True)

if __name__ == "__main__":
    # MODE 1: Local test disabled for live capture
    # test_local()

    # MODE 2: Active connection to your live CTF challenge server!
    HOST = "51.79.140.18"
    PORT = 11040  # <-- Make sure this matches your active port!
    connect_and_solve(HOST, PORT)
```

Flag: `LYKNCTF{53a85fd9fd0742d798f539cbce6adf83}`

End of Write-up